WordPress Security Architecture and Hardening Guide

Enterprise Best Practices and Threat Mitigation

Version 1.1.1 — June 17, 2026

Contents

Table of Contents 3
1. Overview . . . 3
2. Threat Landscape . . . 3
3. WordPress Core Security Architecture . . . 5
3.1 The WordPress Security Team . . . 5
3.2 The Release Cycle . . . 5
3.3 Automatic Background Updates . . . 5
3.4 Backward Compatibility . . . 6
4. OWASP Top 10 Coverage . . . 6
A01:2025 — Broken Access Control . . . 6
A02:2025 — Security Misconfiguration . . . 6
A03:2025 — Software Supply Chain Failures . . . 7
A04:2025 — Cryptographic Failures . . . 7
A05:2025 — Injection . . . 7
A06:2025 — Insecure Design . . . 7
A07:2025 — Authentication Failures . . . 7
A08:2025 — Software or Data Integrity Failures . . . 8
A09:2025 — Security Logging and Alerting Failures . . . 8
A10:2025 — Mishandling of Exceptional Conditions . . . 8
5. Keeping WordPress Up to Date . . . 8
5.1 Recommended Practices . . . 9
6. Server Hardening . . . 9
6.1 Web Server Configuration . . . 9
6.2 Firewall and Network Configuration . . . 10
6.3 PHP and Server-Side Components . . . 11
6.4 File Permissions . . . 11
7. WordPress Application Hardening . . . 12
7.1 Configuration Constants . . . 12
7.2 Disable Unused Features . . . 13
7.3 Database Security . . . 14
7.4 Multisite Security Considerations . . . 14
7.5 REST API Security . . . 15
8. User Authentication and Session Security . . . 16
8.1 Multi-Factor Authentication . . . 16
8.2 Privileged Action Gating . . . 17
8.3 Password Policy . . . 17
8.4 Session Management . . . 18
8.5 Account Management . . . 18
9. Security Plugins and Monitoring . . . 19
9.1 Web Application Firewall . . . 19
9.2 Audit Logging . . . 19
9.3 Malware Detection . . . 20
10. Backup and Recovery . . . 20
11. Supply Chain Security . . . 20
11.1 Software Bill of Materials (SBOM) . . . 21
11.2 Plugin and Theme Management . . . 21
11.3 Internal Toolchain Security . . . 21

11.4 Integrity Verification . . . 22
12. Organizational Security Practices . . . 22
12.1 Employee and Third-Party Access Policies . . . 22
12.2 Security Policies and Governance . . . 22
12.3 Incident Response . . . 23
12.4 Building a Security-First Culture . . . 24
12.5 Privacy and Data Protection . . . 24
13. The Role of the Hosting Provider . . . 25
14. AI Integration Security in WordPress . . . 25
14.1 AI as an Attack Vector . . . 26
14.2 Shadow AI and Governance . . . 26
14.3 Securing AI Integrations in WordPress . . . 27
15. Additional Resources . . . 28
15.1 WordPress Security Documentation . . . 28
15.2 Threat Intelligence and Industry Reports . . . 28
15.3 Standards and Frameworks . . . 29
15.4 Security Culture and Organizational Practices . . . 29
15.5 Deprecated and Invalid Constants Guardrail . . . 29
15.6 Cross-Document Control Classification Matrix . . . 29
Related Documents . . . 30
License and Attribution . . . 31

Table of Contents

1. Overview

WordPress is the dominant content management system, used by 59.8% of websites whose content management system is known and by 42.5% of all websites (W3Techs, March 21, 2026). Licensed under the GPLv2 or later, WordPress benefits from a large, mature development ecosystem and a dedicated security team.

This document provides enterprise hardening guidance for the full WordPress stack — core software, plugin, and theme ecosystem, server environment, and organizational security practices. It provides a comprehensive analysis of the WordPress core software, its security architecture, development processes, and recommended hardening practices. It is intended for developers, system administrators, and technical teams responsible for deploying and maintaining WordPress in enterprise environments.

The security information in this document reflects WordPress core behavior through WordPress 7.1, released on August 19, 2026 (current release as of October 7, 2026: 7.1.3). WordPress 7.2 is scheduled for December 8, 2026 and is not covered. Where a security-relevant feature was introduced in an earlier release, that release is named explicitly. The principles and architectural details described here remain broadly applicable to recent supported versions due to the project's strong commitment to backward compatibility.

> **Guideline Notice** This document supplements organizational vulnerability management standards. It is designed as a hardening guide to reduce the exposed attack surface and provide configuration guidance for WordPress deployments. Questions about this guideline may be directed to your organization's information security team.

2. Threat Landscape

The current threat landscape emphasizes that the most significant risks to WordPress deployments come not from the core software itself but from unpatched third-party components (plugins and themes), misconfigured environments, and compromised user accounts. Patchstack's 2026 State of WordPress Security report, covering 2025 data, found that 91% of newly reported vulnerabilities were in plugins, 9% were in themes, and only 6 low-priority issues were reported in WordPress core.

Patchstack's State of WordPress Security report (2026 edition, covering 2025 data) recorded 11,334 WordPress vulnerabilities — a 42% year-over-year increase following 7,966 in 2024 and 5,948 in 2023. Nearly half (46%) of vulnerabilities had no developer fix at the time of public disclosure, and the weighted median time to first exploit for heavily

exploited vulnerabilities was 5 hours. Patchstack also reported that Broken Access Control was the most exploited vulnerability category in its RapidMitigate telemetry, reinforcing that real-world attack pressure does not map cleanly to raw disclosure counts alone.

The Verizon Data Breach Investigations Report (2025) analyzed over 22,000 security incidents and 12,195 confirmed breaches. It identifies the human element — including errors, social engineering, and misuse — as a contributing factor in approximately 60% of breaches. Credential abuse remains the most common initial access vector (22%), followed by exploitation of vulnerabilities (20%, a 34% year-over-year increase driven largely by edge device and VPN appliance compromises). Third-party involvement in breaches has doubled to 30%, reinforcing the supply chain risks described in Section 11. Ransomware was present in 44% of breaches (up from 32%), though the median ransom payment declined to $115,000 as 64% of victim organizations refused to pay.

IBM's Cost of a Data Breach Report (2025) found the global average breach cost was $4.44 million. Phishing was the most common initial attack vector (16% of breaches, $4.80 million average cost), followed by supply chain compromise (15%, $4.91 million). Organizations with extensive security AI and automation had average breach costs of $3.62 million — $1.88 million less than organizations without — and identified and contained breaches 80 days faster.

Both reports highlight AI as a rapidly growing factor in the threat landscape. The Verizon DBIR found that AI-assisted phishing emails have doubled over the past two years, while IBM reports that 16% of breaches now involve attackers using AI tools (37% for AI-generated phishing, 35% for deepfake-based social engineering). Shadow AI — the unsanctioned use of AI tools by employees — is an emerging cost amplifier: IBM found it added $200,000 to average breach costs, rising to $670,000 for organizations with high shadow AI prevalence, and that 63% of organizations lack AI governance policies. See Section 14 for WordPress-specific GenAI security guidance.

These reports are published annually and should be re-evaluated each year for the latest figures. The primary sources cited in this guide are Patchstack's State of WordPress Security (WordPress-specific vulnerability and exploitation data), the Verizon DBIR (cross-industry breach patterns), and IBM's Cost of a Data Breach Report (financial impact). Additional annual sources worth tracking include the IBM X-Force Threat Intelligence Index (attacker techniques and operational trends), Wordfence's annual threat report (WordPress WAF and scanner telemetry), the Sucuri Website Threat Research Report (WordPress malware and cleanup data), and the CISA Known Exploited Vulnerabilities catalog (continuously updated, increasingly includes WordPress plugin vulnerabilities).

These findings underscore the need for enterprise WordPress teams to adopt robust user management practices, enforce strong authentication, govern the use of AI tools, and cultivate a security-first organizational culture.

3. WordPress Core Security Architecture

3.1 The WordPress Security Team

WordPress core has been actively maintained with security as a first-order concern since the project's founding in 2003. A dedicated Security Team of more than 50 trusted experts — including lead developers, security researchers, and contributors across the project — is responsible for vulnerability triage, patch development, and coordinated disclosure for the core software. The team maintains working relationships with external researchers, hosting companies, and organizations such as HackerOne (see WordPress.org Security page).

Vulnerabilities in WordPress core can be reported through the WordPress HackerOne program. The Security Team follows a responsible disclosure process with severity-based triage and coordinated patch timelines. For a detailed account of the team's structure, history, and processes, see the WordPress Security White Paper.

3.2 The Release Cycle

WordPress follows a major/minor versioning scheme. Major releases (for example, WordPress 7.0 on May 20, 2026 and 7.1 on August 19, 2026, with 7.2 scheduled for December 8, 2026) ship new features on a three-to-four-month cycle through a structured process of scoping, development, beta testing, and release candidates. Minor releases (for example, 6.9.x or 7.0.x) contain maintenance and security fixes. See the WordPress Release Cycle documentation for process details.

3.3 Automatic Background Updates

Automatic background updates for minor security releases have been enabled by default since WordPress 3.7. Only the most recent WordPress release is officially supported; the security team backports fixes to some older branches as a courtesy (as of October 2026, back to 4.7), and that floor moves — security updates for 3.7 through 4.0 ended on December 1, 2022. Enterprise environments should verify this capability remains active and supplement it with managed hosting update pipelines that include staging validation and rollback capability. Disabling automatic updates is possible but strongly discouraged — it shifts the burden of timely patching entirely to the site operator. The pace of security releases shows why: WordPress 7.1.1 (September 17, 2026) and 7.1.3 (October 6, 2026) together fixed 18 security issues within seven weeks of the 7.1 release, including a stored cross-site scripting issue reachable by unauthenticated visitors.

3.4 Backward Compatibility

WordPress prioritizes backward compatibility across major releases, which reduces the risk of breakage during security updates — a key factor for enterprise teams managing large plugin inventories. This commitment lowers the operational cost of staying current and is one of the reasons minor security releases can be safely auto-applied.

4. OWASP Top 10 Coverage

The following describes how WordPress core addresses the OWASP Top 10 Web Application Security Risks (2025 edition).

A01:2025 — Broken Access Control

WordPress provides a granular roles and capabilities system. The core API enforces permission checks before executing any privileged action. Functions like `current_user_can()` verify authorization at the function level. Administrators can further customize roles and capabilities. WordPress uses cryptographic tokens called nonces to validate the intent of action requests and protect against Cross-Site Request Forgery (CSRF). Nonces are scoped to a specific user, action, and time window; they are generated with `wp_create_nonce()` and verified with `wp_verify_nonce()` or `check_ajax_referer()`. Nonces are tied to the current user's session and become invalid if the user logs in or out. Plugins and themes that handle form submissions or AJAX requests must implement nonce verification — missing or broken nonce checks are among the most common plugin vulnerability patterns. Safe HTTP request wrappers such as `wp_safe_remote_get()` validate URLs with `wp_http_validate_url()` to reduce Server-Side Request Forgery (SSRF) exposure by rejecting unsafe hosts and ports. For defense in depth, supplement these application-layer checks with network-level egress filtering and WAF rules.

A02:2025 — Security Misconfiguration

WordPress provides configuration constants (in `wp-config.php`) to harden installations: `DISALLOW_FILE_EDIT`, `DISALLOW_FILE_MODS`, `FORCE_SSL_ADMIN`, and others. The core team publishes documentation and best practices for secure server configuration.

A03:2025 — Software Supply Chain Failures

The core team monitors and updates bundled libraries (jQuery, TinyMCE, PHPMailer, etc.). Automatic background updates ensure core patches reach sites promptly. The plugin/theme repository team reviews submissions and can remove or update vulnerable components. See also Section 11 (Supply Chain Security) for extended guidance on managing the WordPress plugin and theme ecosystem.

A04:2025 — Cryptographic Failures

As of WordPress 6.8, passwords are hashed with bcrypt (SHA-384 pre-hashed to work around bcrypt's 72-byte input limit), and security tokens — including application passwords and password reset keys — use BLAKE2b via libsodium. On supported WordPress 7.0 environments (PHP 7.4+), sites with Argon2 support can enable Argon2id hashing via the `wp_hash_password_algorithm` filter for even stronger resistance to brute-force and GPU-accelerated attacks. WordPress supports HTTPS enforcement through configuration constants and provides salting via security keys defined in `wp-config.php`. Sensitive data like user email addresses and private content is access-controlled through the permissions system.

A05:2025 — Injection

WordPress provides the `$wpdb->prepare()` method for parameterized database queries, preventing SQL injection. Input sanitization and output escaping functions (`esc_html()`, `esc_attr()`, `wp_kses()`, etc.) are available throughout the API. File upload restrictions limit the types of files that can be uploaded.

A06:2025 — Insecure Design

WordPress core follows security-by-default principles. Default settings are evaluated by the core team for security implications. The REST API requires authentication for sensitive endpoints. The block editor (Gutenberg) sanitizes content at multiple levels.

A07:2025 — Authentication Failures

WordPress handles authentication entirely server-side — passwords are bcrypt-hashed (see A04 above) and session tokens are invalidated on logout. The platform supports application passwords for REST API and XML-RPC authentication — these are individually named and revocable credentials that are not valid for Dashboard login. They are not

permission-scoped: each application password carries the full capabilities of the user who owns it. They also bypass 2FA and should be managed carefully (see Section 8). WordPress is compatible with two-factor authentication plugins.

A08:2025 — Software or Data Integrity Failures

WordPress includes integrity-verification mechanisms for downloaded packages and installed files, and enterprises can supplement them with checksum validation and version-controlled deploys. These controls reduce the risk of tampered code reaching production, but they should be treated as one layer in a broader software integrity program rather than as a complete supply-chain guarantee on their own.

A09:2025 — Security Logging and Alerting Failures

While WordPress core provides limited built-in logging, the ecosystem offers robust audit logging solutions (e.g., WP Activity Log). Enterprise hosting platforms typically provide comprehensive server-level logging, SIEM integration, and monitoring.

A10:2025 — Mishandling of Exceptional Conditions

WordPress core includes structured error handling through the `WP_Error` class and provides mechanisms to control error output in production environments (`WP_DEBUG`, `WP_DEBUG_DISPLAY`, `WP_DEBUG_LOG`).

5. Keeping WordPress Up to Date

> **Key Principle** Only the latest major version of WordPress receives new features and full development support. However, the security team backports critical security patches to a range of older branches as a courtesy, not a commitment (as of October 2026, back to WordPress 4.7; updates for 3.7 through 4.0 ended on December 1, 2022). An older branch that still receives backports is not a supported version. Keeping WordPress core, all plugins, and all themes up to date remains the single most important security measure for any WordPress deployment.

Unpatched software is the most common technical root cause of WordPress compromises. Vulnerability databases consistently show that outdated plugins with known, publicly disclosed vulnerabilities are the primary attack vector.

5.1 Recommended Practices

- Enable automatic background updates for WordPress core (enabled by default since 3.7).
- Establish a regular maintenance cycle for plugin and theme updates, with staging environment testing.
- Subscribe to security advisory feeds from Patchstack, WPScan, or Wordfence to receive early notification of vulnerabilities. Use the Exploit Prediction Scoring System (EPSS) probability alongside CVSS severity to prioritize remediation by real-world exploitability, not theoretical severity alone. EPSS scores are increasingly reported by Patchstack and other databases alongside CVSS.
- Remove unused plugins and themes. Deactivated code can still be exploited if accessible on the server.
- Use managed WordPress hosting that provides automatic patching with rollback capabilities.
- Deploy virtual patching (e.g., via Patchstack or Cloudflare WAF rules) when a plugin security update cannot be immediately applied.

6. Server Hardening

The configuration of the underlying server and hosting environment is as important as the WordPress application itself. A misconfigured server can expose even a fully patched WordPress installation to compromise.

6.1 Web Server Configuration

The web server (Nginx or Apache) serves as the first line of defense. Organizations should follow prescriptive hardening benchmarks such as the CIS Benchmarks for web servers and WordPress.

Each recommendation in this section corresponds to an auditable control in the WordPress Security Benchmark §1.1 through §1.5, which provide step-by-step audit commands, remediation snippets, and default-value documentation.

- **Enforce TLS 1.2+:** Disable legacy support for TLS 1.0 and 1.1. Only TLS 1.2 and 1.3 should be accepted to mitigate protocol-level attacks like BEAST and POODLE.
- **Hide Server Tokens:** Configure the web server to suppress version numbers and operating system information in HTTP headers and error pages (`server_tokens off` in Nginx; `ServerTokens Prod` and `ServerSignature Off` in Apache).

- **HTTP Security Headers:** Implement a robust set of security headers to instruct the browser to enable built-in protections:
  - `Content-Security-Policy` (CSP): Restrict sources of scripts, styles, and other resources. Level 2 configurations should aim to remove `unsafe-inline` through the use of nonces or hashes. From WordPress 7.1, the block editor's client-side media processing runs in a Web Worker loaded from a `blob:` URL; a policy without `worker-src 'self' blob:` blocks it and WordPress falls back to server-side processing.
  - `X-Content-Type-Options`: Set to `nosniff` to prevent MIME-type confusion.
  - `X-Frame-Options`: Set to `SAMEORIGIN` or `DENY` to protect against clickjacking.
  - `Strict-Transport-Security` (HSTS): Enforce HTTPS for a specified duration (e.g., one year).
  - `Referrer-Policy`: Set to `strict-origin-when-cross-origin` to limit referrer leakage.
  - `Permissions-Policy`: Restrict browser features like geolocation, camera, and microphone.
- **Block PHP Execution in Uploads:** Explicitly deny PHP processing in the `wp-content/uploads/` directory to prevent the execution of malicious files uploaded through potential vulnerabilities.
- **Rate Limiting:** Implement rate limiting at the web server level for `wp-login.php`, `xmlrpc.php`, and the REST API (`/wp-json/`) to throttle automated brute-force and resource exhaustion attempts.

6.2 Firewall and Network Configuration

- Deploy a host-based firewall (e.g., UFW on Ubuntu/Debian) restricting inbound traffic to required ports only (typically 80, 443, and SSH).
- Implement Fail2Ban to detect and block malicious patterns at the server level, including integration with WordPress login logs.
- Maintain IP denylists (e.g., 7G/8G rulesets) to filter known malicious traffic and bad bots.
- Deploy a Web Application Firewall (WAF) at the network edge (e.g., Cloudflare) or on the server (e.g., ModSecurity 3+ with the OWASP Core Rule Set).

6.3 PHP and Server-Side Components

- Keep PHP on an actively supported version. WordPress.org currently recommends PHP 8.3 or greater. Standardize production deployments on PHP 8.3+ and validate PHP 8.4 in staging before production rollout.
- Harden the PHP runtime: set `expose_php = Off` to prevent version disclosure in HTTP headers, set `display_errors = Off` and `log_errors = On` in production to prevent leaking file paths and database details, disable dangerous functions via `disable_functions` (e.g., `exec`, `passthru`, `shell_exec`, `system`, `proc_open`, `popen`), and restrict PHP file operations with `open_basedir` to the WordPress installation directory and required system paths. The first three are baseline (Benchmark Level 1) controls; `open_basedir` is an optional, environment-specific (Level 2) control and, per the PHP manual, is not a complete security boundary. For high-security environments, consider the Snuffleupagus PHP security extension to mitigate `eval()` and provide additional hardening beyond `disable_functions`.
- Configure PHP session security: set `session.cookie_secure = 1`, `session.cookie_httponly = 1`, `session.cookie_samesite = Lax`, `session.use_strict_mode = 1`, and `session.use_only_cookies = 1`. Note: WordPress core does not use PHP native sessions — these settings are defense-in-depth for plugins that call `session_start()`.
- Keep all server-side components (web server, database server, operating system) on supported, actively maintained versions.
- Achieve an A+ grade on TLS configuration assessments (e.g., Qualys SSL Labs) by using modern cipher suites and disabling legacy protocols.
- Require SSH key-based authentication; disable password-based SSH access.
- Prefer SFTP or SCP over legacy FTP. If FTPS is used for compatibility, enforce TLS and strong credentials, and plan migration to SFTP.
- Enforce per-site process isolation in containerized or chroot environments.
- Consider placing `wp-config.php` above the document root where server configuration allows. This is defense-in-depth and must be validated carefully; a misconfigured web root can reduce security.

6.4 File Permissions

Restrict file permissions and ownership using a documented least-privilege model. File permissions are context-dependent; the WordPress Security Style Guide §3.7 explains

how to communicate environment-specific recommendations. The WordPress Security Benchmark §6.1 provides the auditable control.

Ownership model (choose and document per environment):

- **Model A (preferred on dedicated/self-managed hosts):** site-owned files with web-server group read access (for example, `wp_user:www-data`), plus controlled write access only where needed (uploads/cache).
- **Model B (provider-constrained shared/managed hosting):** provider-required ownership model with compensating controls (`DISALLOW_FILE_EDIT`, controlled update process, and stricter monitoring).

Recommended permissions:

- Directories: 755 (or 750 where group/world read is not required).
- Files: 644 (or 640).
- `wp-config.php`: 400 or 440 preferred steady-state; 600/640 may be used temporarily when deployment automation must write, then revert.
- Set `DISALLOW_FILE_EDIT` to `true` in `wp-config.php` as the baseline to disable the built-in editor. Use `DISALLOW_FILE_MODS` only in hardened profiles with a documented external update pipeline.

---

7. WordPress Application Hardening

7.1 Configuration Constants

Set the following security-related constants in `wp-config.php`:

- `DISALLOW_FILE_EDIT` — Disables the built-in theme and plugin editor in the Dashboard.
- `DISALLOW_FILE_MODS` — Optional hardened profile: prevents plugin/theme uploads and updates through the Dashboard; requires an external update process.
- `FORCE_SSL_ADMIN` — Forces HTTPS on all admin and login pages.
- `WP_AUTO_UPDATE_CORE` — Controls automatic core updates (set to `true` or `'minor'`).

- `WP_DEBUG` — Must be `false` in production. Set `WP_DEBUG_DISPLAY` to `false` as well. If `WP_DEBUG_LOG` is enabled, direct the log to a non-public path (e.g., `/var/log/wordpress/debug.log`) to prevent exposure of file paths, database queries, and PHP errors.
- **Authentication Keys and Salts** — All eight authentication keys and salts (`AUTH_KEY`, `SECURE_AUTH_KEY`, `LOGGED_IN_KEY`, `NONCE_KEY`, and their corresponding `_SALT` counterparts) must be set to unique, random values. Generate them via `curl -s https://api.wordpress.org/secret-key/1.1/salt/`. Placeholder values (`'put your unique phrase here'`) must be replaced before deployment.

7.2 Disable Unused Features

- **Disable XML-RPC** if not required. It is a second password-authentication surface, and its pingback methods have been abused for DDoS reflection. (The `system.multicall` brute-force amplification technique is historical: since WordPress 4.4, core stops evaluating credentials for the rest of a request after the first failed login.) WordPress core disables the loading of custom XML entities to prevent XML eXternal Entity (XXE) and entity expansion attacks, but disabling XML-RPC entirely removes the endpoint from the attack surface. Block `xmlrpc.php` at the web server level; this is the only complete block. The `xmlrpc_enabled` filter (`add_filter( 'xmlrpc_enabled', '__return_false' )`) is a partial measure: it turns off only the methods that require authentication and leaves pingbacks and other unauthenticated methods available. Do not rely on `wp-config.php` constants — `XMLRPC_REQUEST` is a read-only internal constant that WordPress sets during XML-RPC processing and cannot be used to disable the feature.
- **Disable trackbacks and pingbacks** in **Settings Discussion**. Trackbacks operate independently of `xmlrpc.php` and must be disabled separately.
- Disable the built-in file editor.
- Prevent username enumeration via the REST API and author archives. Closing those routes does not change the names WordPress publishes in bylines, feeds and comments; see §8.5 for public author identities.
- Scope unauthenticated REST API access to only the public endpoints your site requires (e.g., posts listing for a public front-end). Avoid blanket API blocking unless architecture explicitly requires it.
- For higher reliability, consider replacing built-in `wp-cron.php` triggers with a system cron job: set `define( 'DISABLE_WP_CRON', true );` in `wp-config.php` and add a system cron entry (for example, `*/5 * * * * cd /path/to/wordpress`

`&& wp cron event run --due-now`). Without the constant, the system cron runs in addition to page-load triggers rather than replacing them. Treat this primarily as an operations/reliability control, with secondary security benefits.

7.3 Database Security

For prescriptive audit and remediation procedures corresponding to each database recommendation below, see WordPress Security Benchmark §3.1 through §3.4.

- Database table prefixes are a low-value obscurity control. Use a non-default prefix only as optional defense-in-depth; prioritize patching, least privilege, and secure coding controls.
- Grant the database user only the minimum required privileges: SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, and DROP on the WordPress database only. CREATE, ALTER, INDEX, and DROP are needed for plugin table creation, schema updates, and core updates. Some plugins may also require CREATE TEMPORARY TABLES or LOCK TABLES — add only when verified necessary.
- Configure MySQL/MariaDB to listen only on localhost (`bind-address = 127.0.0.1`) or a Unix socket. Remote TCP connections should be disabled unless required and tunneled through SSH or a VPN.
- Enable slow query logging (`slow_query_log = 1`) for forensic analysis and intrusion detection. General query logging incurs significant I/O overhead and should be used selectively or only during investigations.
- Encrypt sensitive data stored in the database, including API keys, SMTP credentials, and payment gateway tokens.
- Use a dedicated database user per WordPress installation.

7.4 Multisite Security Considerations

WordPress Multisite enables a single WordPress installation to serve a network of sites from a shared codebase and database. This architecture introduces additional security considerations:

- **Super Admin Role:** The Super Admin role has unrestricted access across the entire network. Limit the number of Super Admin accounts and treat them as the highest-privilege tier in your access control policies.

- **Network-Level Plugin and Theme Control:** Only Super Admins can install, activate, or remove plugins and themes at the network level. Site-level administrators cannot install new code, which reduces the attack surface but concentrates privilege.
- **Shared Database Tables:** All sites in a Multisite network share user and metadata tables. A compromise of one site's administrator account can potentially affect the entire network if combined with privilege escalation.
- **Cross-Site Attack Surface:** Plugins activated network-wide run on every site. A vulnerability in a network-activated plugin exposes all sites simultaneously.
- **Domain Mapping and TLS:** When using domain mapping for subsites, ensure each mapped domain has valid TLS certificates and appropriate security headers.
- **Configuration Gating:** In Multisite environments, apply reauthentication requirements (Section 8.2) at the network level for Super Admin actions such as adding sites, managing network-wide plugins, and modifying network settings.

7.5 REST API Security

The WordPress REST API (`/wp-json/`) provides a structured interface for applications to interact with WordPress over HTTP. It powers the block editor, mobile apps, headless front-ends, and third-party integrations. Because of its broad surface area, securing the REST API is essential for any enterprise deployment.

- **Authentication Methods:** The REST API supports cookie-based authentication (with nonce validation for CSRF protection), application passwords (individually revocable credentials that do not grant Dashboard login access but carry the owning user's full capabilities — issue them from a dedicated least-privilege integration account, not an administrator's), and extensible authentication via plugins (OAuth 2.0, JWT, etc.). Choose the method appropriate to each integration and enforce the principle of least privilege.
- **Permission Callbacks:** Every REST API endpoint includes a `permission_callback` that determines whether the current user is authorized to perform the requested action. Custom endpoints must always implement permission checks — endpoints without them are publicly accessible by default.
- **Restrict Public Exposure:** By default, some REST API endpoints expose information about users, posts, and site structure to unauthenticated requests. Restrict or disable endpoints that are not required for public consumption (see Section 7.2). For headless or decoupled architectures, allowlist only the specific routes needed by the front-end application.

- **Data Validation and Sanitization:** REST API endpoints validate input against JSON Schema definitions, isolating invalid data before it reaches callback functions. Custom endpoints should define schemas for all arguments and use the built-in validation and sanitization infrastructure rather than manual parsing.
- **CORS (Cross-Origin Resource Sharing):** The REST API sends default CORS headers, but WordPress does not verify the incoming `Origin` header on public REST requests. CSRF protection for cookie-authenticated requests relies on REST nonces instead. If your architecture requires stricter cross-origin controls (for example, a decoupled front-end on a separate domain), replace the default behavior with an explicit origin allowlist rather than using a wildcard (*).
- **Rate Limiting:** Apply rate limiting to REST API routes at the web server level (see Section 6.1) to prevent abuse, enumeration, and resource exhaustion attacks.

8. User Authentication and Session Security

User authentication and session management represent the most critical—and most frequently exploited—aspects of WordPress security. Compromised user credentials and hijacked sessions are among the most common ways WordPress sites are breached.

> **Current Threat Context** The Verizon DBIR (2025) found that credential abuse remains the most common initial access vector (22% of breaches), and the SpyCloud Annual Identity Exposure Report (2025) confirms the scale of the underlying problem: over 750 million credentials were exposed in the past year, with infostealer malware responsible for a growing share. Session hijacking, credential stuffing, and infostealer malware are prominent and growing attack categories across web platforms.

8.1 Multi-Factor Authentication

- Require multi-factor authentication (MFA) or two-factor authentication (2FA) for all administrator and editor accounts.
- Standardize on `two-factor` for operational consistency across environments, unless an approved equivalent is required by platform constraints. The plugin provides enrollment and the login challenge only; it has no built-in setting that requires 2FA for a role, so mandatory 2FA needs a separate enforcement control (for example, MFA enforced at the identity provider).
- Use TOTP-based authentication apps (e.g., Authy, Google Authenticator), hardware security keys (WebAuthn/FIDO2), or passkeys for phishing-resistant passwordless

authentication. Passkey support in WordPress core is anticipated in a future release; plugins currently provide this capability.

- Do not use SMS-based 2FA, as it is vulnerable to SIM-swapping attacks.
- Protect 2FA secrets at rest, and state which layer does it. The `two-factor` plugin stores TOTP secrets unencrypted in user meta, so anyone who can read the database or a database backup can read them. Volume or database encryption protects against stolen disks, not against SQL injection or an exposed backup. Where the threat model requires field-level encryption, use an approved implementation that provides it; otherwise treat database access and backups as holding MFA secrets and protect them accordingly.
- Encourage all users, including contributors and subscribers, to enable 2FA.

> For the operational procedure to install, configure, verify, and provide emergency recovery for the `two-factor` plugin via WP-CLI, see WordPress Operations Runbook §5.5.

8.2 Privileged Action Gating

Enterprise environments should implement action-gated reauthentication for high-risk operations. (Also known as "sudo mode." See the WordPress Security Style Guide Glossary for the formal definition and usage guidance.) This requires users—even those already logged in with administrative privileges—to reconfirm their identity before performing sensitive tasks.

Recommended gated actions include: - Installing, activating, or deleting plugins and themes. - Modifying `wp-config.php` or other critical system settings. - Creating or promoting user accounts to administrative roles. - Executing WordPress core updates or downgrades. - Exporting site data.

This secondary layer of authentication mitigates the risk of session hijacking, as a stolen session cookie alone is insufficient to perform destructive actions. Managed platforms may provide this capability natively — for example, WordPress VIP step-up authentication requires MFA reauthentication before accessing higher-risk resources or performing sensitive, irreversible VIP Dashboard actions, with a one-hour unlock window.

8.3 Password Policy

- Enforce strong passwords of at least 15 characters as the baseline recommendation, following NIST SP 800-63B Rev. 4 guidance. NIST permits 8 characters only when a password is used solely as part of a multi-factor authentication flow.

- Block passwords found in known breach databases (e.g., Have I Been Pwned).
- Do not enforce arbitrary complexity rules (e.g., requiring special characters) that encourage predictable patterns; enforce length and entropy instead.
- On servers with the necessary PHP extensions, consider enabling Argon2id password hashing via the `wp_hash_password_algorithm` filter for stronger resistance to GPU-accelerated brute-force attacks.

8.4 Session Management

- Enforce short maximum session lifetimes (8-24 hours for privileged users).
- Disable or minimize the "Remember Me" option for administrator accounts.
- Automatically terminate idle sessions after a defined inactivity period.
- Terminate all active sessions daily at scheduled times, or on role/permission changes.

8.5 Account Management

> **Principle of Least Privilege** Grant users access to the minimum level of permissions they need to perform their functions. Review and audit user roles regularly. Remove accounts that are no longer needed.

- Limit the number of administrator accounts. Reserve the primary admin for emergency "break glass" scenarios.
- Whoever publishes gets an Author or Editor account, and the administrator account never publishes or comments while logged in. Bylines, feeds, comments and author URLs name the publishing account, so this keeps the public record pointing at accounts that cannot take over the site (see Benchmark §5.2).
- Keep public author identities separate from credentials. WordPress fills an account's display name, nickname and URL slug from its login unless told otherwise, so an email-address login ends up published in bylines, feeds, comments and author URLs. Every account also accepts its email address at login. Treat the login as private (an email address is fine), set the display name to the byline, and build the slug from the byline (see Benchmark §5.9).
- Create custom roles with only the capabilities each user group requires.
- Keep the intended role and capability definitions in a version-controlled must-use plugin and reconcile the stored definitions against it. WordPress still stores roles in the database, and `add_role()` does nothing when a role already exists, so code alone does not prevent tampering; it provides an auditable source of truth that makes tampering detectable and reversible. User-to-role assignments remain database-backed and need separate review (see Benchmark §5.8). (The

WordPress roles API is not available during `wp-config.php` loading — roles must be registered on the `init` hook or later.)
- Restrict administrator capabilities such as file upload, plugin/theme installation, and code editing by default.
- Implement IP or device-based allowlists for privileged accounts where feasible.
- Adopt trusted device verification for accounts with elevated privileges.
- Immediately revoke access for departed employees and terminated third-party contractors.

9. Security Plugins and Monitoring

9.1 Web Application Firewall

Deploy a WordPress-aware WAF that provides:

- Real-time threat intelligence feeds and virtual patching.
- Protection against common attack patterns (SQLi, XSS, CSRF, file inclusion).
- Brute-force protection with intelligent rate limiting.
- Bot detection and management.

Options include Patchstack, Wordfence, Sucuri, and Cloudflare (at the network edge).

9.2 Audit Logging

- Install a comprehensive audit logging plugin (e.g., WP Activity Log) that records all user activity, including logins, content changes, plugin/theme modifications, and settings changes.
- Retain logs for a period consistent with your organization's compliance requirements.
- Configure log alerts for suspicious activity: failed logins, privilege escalation, file modifications, and new user account creation.
- Export logs to a centralized SIEM system for correlation with other security events.

9.3 Malware Detection

- Deploy server-level malware detection (e.g., Imunify360, Linux Malware Detect, ClamAV).
- Schedule regular integrity checks comparing core files against known-good checksums.
- Monitor for unauthorized file changes, especially in plugin and theme directories.

10. Backup and Recovery

Robust backup and recovery capabilities are essential. In the event of a security breach, the most reliable recovery strategy is to identify the root cause, verify the integrity of backups, and rebuild the compromised system from a known-good state.

- Perform backups at the server level (not relying solely on WordPress plugins).
- Store backups offsite, in a location inaccessible from the production environment.
- Encrypt backup data both in transit and at rest.
- Test backup restoration procedures regularly (at least quarterly).
- Maintain multiple backup generations with sufficient retention to recover from undetected compromises.
- Document the recovery procedure and assign clear ownership.

11. Supply Chain Security

WordPress's extensibility through plugins and themes introduces supply chain risk. Unlike sandboxed extension models found in some platforms, WordPress's plugin architecture executes all third-party code at the same privilege level as core, with full access to the database, filesystem, and WordPress APIs. There is no built-in capability isolation between plugins. This design maximizes flexibility and performance but amplifies the impact of any single compromised or vulnerable component, making the vetting and management of plugins and themes a critical security concern.

The scale of this risk is growing. The Verizon DBIR (2025) found that third-party involvement in breaches has doubled to 30%, driven in part by exploitation of software supply chain dependencies and partner-connected access. IBM's Cost of a Data Breach Report

(2025) found supply chain compromise to be the second most common initial attack vector (15% of breaches) with an average cost of $4.91 million — the highest cost amplifying factor across all breach categories. The Verizon report also found that exploitation of vulnerabilities in edge devices and VPN appliances increased from 3% to 22% of vulnerability-related breaches, with a median remediation time of 32 days and only 54% of affected devices fully patched during the reporting period. These trends are directly relevant to WordPress environments that rely on third-party plugins, themes, and hosting infrastructure.

11.1 Software Bill of Materials (SBOM)

In response to increasing software supply chain attacks, enterprise organizations should maintain a Software Bill of Materials (SBOM) for their WordPress deployments. An SBOM is a formal, machine-readable inventory of all software components, their versions, and their relationships.

For WordPress, a comprehensive SBOM should include: - WordPress core version. - All active and inactive plugins and themes. - Third-party libraries bundled with plugins/themes (e.g., jQuery, PHPMailer). - PHP version and loaded extensions. - Web server and database versions.

Maintaining an SBOM allows for rapid impact assessment when a new vulnerability is disclosed in a common component or library.

> The WordPress Security Benchmark §8.4 provides an auditable control for SBOM maintenance, including tooling recommendations and CI/CD integration guidance.

11.2 Plugin and Theme Management

- Only install plugins and themes from trusted sources (WordPress.org repository, reputable commercial vendors).
- Evaluate plugins for active maintenance, update frequency, known vulnerabilities, and code quality before deployment.
- Remove all unused plugins and themes from the server (deactivation alone is insufficient).
- Monitor plugin vulnerability disclosures and apply patches promptly.

11.3 Internal Toolchain Security

- Verify the integrity of build and deployment tools.
- Use version-controlled, auditable deployment pipelines.

- Pin dependency versions and verify checksums for all external packages.
- Conduct code reviews for custom plugins and theme code before deployment.

11.4 Integrity Verification

Implement automated integrity checks to verify that the code on the production server matches the version-controlled source or the official WordPress.org checksums. Any unauthorized file changes should trigger immediate alerts.

12. Organizational Security Practices

Technical controls alone are insufficient. The human element accounts for the majority of security incidents. Organizations must complement technical hardening with policies, training, and cultural practices.

12.1 Employee and Third-Party Access Policies

- Require 2FA and VPN for all remote access to WordPress admin interfaces.
- Require timely OS and software updates on all devices used to access WordPress.
- Require email scanning for malware and endpoint protection software.
- Address phishing and social engineering in employee onboarding and recurring training.
- Define and enforce a BYOD policy or restrict administrative access to managed devices.
- Terminate access promptly when employees or contractors depart.

12.2 Security Policies and Governance

- Define and enforce a written user security policy covering password standards, session management, and acceptable use.
- Adopt a Zero-Trust model: continuously verify active users regardless of network location.
- Establish software version management and update policies with defined SLAs.
- Define security metrics and conduct regular audits (internal and external).

- Create, document, and practice an incident response plan with assigned roles (see Section 12.3).
- Maintain a disaster recovery plan integrated with the business continuity plan.
- Require written SLAs with hosting providers and third-party data handlers that address security, privacy, and compliance.
- Establish an AI governance policy covering approved tools, acceptable use, data classification for AI inputs, and authentication requirements. IBM's Cost of a Data Breach Report (2025) found that 63% of organizations lack AI governance policies and that shadow AI incidents added $200,000 to average breach costs ($670,000 for organizations with high shadow AI prevalence). See Section 14 for implementation guidance.

12.3 Incident Response

Every enterprise WordPress deployment should have a documented incident response plan. A structured approach reduces recovery time and limits damage. Follow an established framework such as NIST SP 800-61r3 (Section 3 provides the core incident handling lifecycle):

1. **Preparation:** Maintain response playbooks, define roles and communication channels, and verify that logging and monitoring are operational.
2. **Identification:** Detect incidents through WAF alerts, integrity monitoring, audit logs, user reports, or external vulnerability disclosures. Determine the scope: which sites, users, and data are affected.
3. **Containment:** Isolate the affected site or server. Revoke compromised credentials. Enable maintenance mode. Preserve forensic evidence (logs, modified files, database snapshots) before making changes.
4. **Eradication:** Remove malicious code, close the attack vector (patch the vulnerability, remove the compromised plugin), and verify file integrity against known-good checksums or version control.
5. **Recovery:** Restore from a verified clean backup if necessary. Redeploy from version-controlled source code. Force password resets for all affected accounts. Re-enable the site and monitor closely for recurrence.
6. **Lessons Learned:** Conduct a post-incident review within 72 hours. Document root cause, timeline, impact, and remediation steps. Update security policies, monitoring rules, and response playbooks based on findings.

> For hands-on-keyboard procedures implementing each phase of this lifecycle — containment commands, forensic artifact capture, malware scanning, credential reset, and recovery validation — see WordPress Operations Runbook §10.3 and §10.4.

12.4 Building a Security-First Culture

Industry analysts (Gartner Security and Risk Management Summit, 2024) emphasize that third-party breaches are inevitable and that organizations should prioritize resilience over prevention alone. IBM's Cost of a Data Breach Report (2025) provides the supporting data: 65% of breached organizations reported they had not fully recovered. Organizations should focus on fostering behavioral change over mere awareness:

- Train teams with simulated breach scenarios and tabletop exercises.
- Make security practices habitual, not just policy documents.
- Ensure norms, values, and assumptions across the organization align with security goals.
- Empower all team members to identify and report potential compromise.

12.5 Privacy and Data Protection

WordPress deployments that collect, store, or process personal data must comply with applicable data protection regulations such as GDPR, CCPA/CPRA, and other regional frameworks.

- **Data Minimization:** Collect only the personal data necessary for the stated purpose. Audit plugins and forms for unnecessary data collection.
- **Privacy Tools:** WordPress core (since version 4.9.6) includes built-in privacy tools: a privacy policy page generator, personal data export, and personal data erasure request handling. Use these tools to respond to data subject access requests.
- **Consent Management:** Implement cookie consent and data processing consent mechanisms that comply with applicable regulations. Ensure consent records are auditable.
- **Data Encryption:** Encrypt personal data at rest in the database and in transit via TLS. Pay particular attention to form submissions, user metadata, and WooCommerce or membership plugin data.
- **Third-Party Data Sharing:** Audit all plugins and integrations that transmit data to external services (analytics, marketing, CDN, AI/LLM providers). Maintain a data processing agreement with each third-party service.

- **Retention Policies:** Define and enforce data retention schedules. Automatically purge data that is no longer needed for its stated purpose.

13. The Role of the Hosting Provider

WordPress can be installed on virtually any server environment, but the hosting infrastructure is a critical security layer. Enterprise deployments should require:

- Per-site process isolation in containerized or chroot environments.
- Managed, automated patching for the full server stack (OS, PHP, database, web server).
- Multiple upstream security layers (network-level DDoS mitigation, WAF, intrusion detection).
- Automated, offsite backups with tested recovery procedures.
- Relevant assurance evidence: ISO/IEC 27001 certification, a current SOC 2 attestation report, PCI DSS validation, GDPR-aligned data processing agreements, and for government/education, FedRAMP authorization or equivalent.
- An immutable filesystem where applicable, preventing runtime file modifications.

Leading enterprise WordPress hosts hold ISO/IEC 27001 certification, publish SOC 2 attestation reports, and maintain PCI DSS validation. WordPress VIP additionally holds FedRAMP authorization for United States federal projects. These are different kinds of evidence: a certification, an auditor's report on a defined system and period, a validation against a payment standard, and a government authorization. When evaluating hosting providers, check the scope and date of each and verify that it matches your organization's compliance requirements.

14. AI Integration Security in WordPress

As organizations integrate Generative AI (GenAI) into their WordPress workflows — for content generation, chat interfaces, and automated site management — new security considerations emerge. These risks are no longer theoretical: IBM's Cost of a Data Breach Report (2025) found that 13% of organizations experienced a breach involving an AI model or application, and 97% of those breaches involved AI systems lacking proper access controls. The most common AI-specific attack types were supply chain compromise of AI components (30%), model inversion (24%), model evasion (21%), prompt injection (17%), and data poisoning (15%).

WordPress 7.0 makes this discussion more concrete by shipping first-party AI integration infrastructure, including the `WP AI Client`, the client-side `Abilities API`, the `AI Connectors` screen, and the `Connectors API`. See the WordPress 7.0 Field Guide and

the developer notes for the AI Client, the Client-Side Abilities API, and the Connectors API. For enterprise teams, these are not merely convenience features; they create new secret-management, authorization-boundary, and provider-governance decisions that should be handled as part of the platform security model.

14.1 AI as an Attack Vector

AI tools are increasingly weaponized by threat actors. The Verizon DBIR (2025) found that AI-assisted malicious emails have doubled over the past two years. IBM reports that 16% of breaches now involve attackers using AI, with 37% employing AI-generated phishing and 35% using deepfake-based social engineering. For WordPress sites, this means:

- **AI-enhanced phishing and social engineering** targeting WordPress administrators and users will be more convincing and harder to detect. Training and awareness programs must account for AI-generated content.
- **Automated vulnerability discovery** using AI may accelerate the exploitation window for WordPress plugin vulnerabilities, increasing the urgency of timely patching and virtual patching.

14.2 Shadow AI and Governance

Shadow AI — the unsanctioned use of AI tools by employees — is an emerging organizational risk. IBM found that 20% of breached organizations experienced a shadow AI-related incident (distinct from the 16% of breaches involving attacker AI use in Section 14.1), adding $200,000 to average breach costs ($670,000 for organizations with high shadow AI prevalence). The Verizon DBIR found that 15% of employees routinely access GenAI systems on corporate devices (at least once every 15 days), with 72% using non-corporate email accounts and only 17% using corporate email with integrated authentication.

For WordPress teams, shadow AI risks include content contributors pasting sensitive draft content into public AI tools and developers using AI code assistants that may introduce vulnerabilities or leak proprietary code. Organizations should establish an AI acceptable use policy, maintain an inventory of approved AI tools, and enforce authentication controls on any AI service used in the content workflow.

14.3 Securing AI Integrations in WordPress

When WordPress 7.0 AI infrastructure is in scope, treat connectors, prompt execution paths, and abilities registration as security-relevant application surfaces, not just developer ergonomics.

- **Data Privacy:** Ensure that sensitive site data or user information is not inadvertently sent to LLM providers during prompt processing. Use private or enterprise-tier AI services that guarantee data will not be used for model training.
- **Prompt Injection:** Input validation reduces prompt injection but cannot prevent it (see OWASP LLM01:2025). Instructions can arrive as ordinary, well-formed text — typed by a user (direct injection) or carried in content the model retrieves, such as posts, comments, uploaded files, or web pages (indirect injection). Design on the assumption that the model can be manipulated: authorize every tool call and data access in deterministic code using the requesting user's capabilities, give AI integrations least-privilege credentials, keep the set of available operations narrow, and require human approval for high-impact actions. This applies to AI-powered chatbots, search features, and content generation tools integrated with WordPress.
- **Output Handling:** Treat GenAI-generated content as untrusted user input. Sanitize and escape AI output before displaying it on the site. Escaping does not make generated code safe to run: code or commands produced by a model (for example, in automated site management tools) need human review, validation against an allowlist of permitted operations, and execution in an isolated, least-privilege environment.
- **Access Controls for AI Systems:** Implement proper authentication and authorization for all AI model endpoints and APIs. IBM found that 97% of AI-related breaches involved systems lacking proper access controls — apply the same role-based access control principles used for WordPress itself to any AI integrations.
- **Client-Side Prompt Execution:** Treat arbitrary browser-side prompt execution as a high-privilege pattern. The JavaScript AI client wrapper described in the WordPress 7.0 AI Client dev note is distributed separately from core; it uses REST endpoints under the hood and is intentionally restricted because it can send arbitrary prompts to configured providers. For general plugin and site features, prefer narrow server-side REST endpoints with granular permission checks, and keep prompt construction, provider selection, and connector configuration scoped server-side.
- **Abilities API Boundaries:** Treat the Abilities API as an authorization surface. Abilities exposed through `/wp-abilities/v1/` should follow the same least-privilege design expectations as any other capability-sensitive REST integration, especially where AI agents or workflow tools are involved. WordPress 7.1 adds two things architects should account for. First, a `public` metadata flag (default `false`) now controls whether an ability is exposed to external clients; the dev note is explicit that exposure flags are not a security boundary, so

authorization must live in each ability's `permission_callback`. Second, execution lifecycle filters let any active plugin overturn a denial on every execution path, including REST (`wp_ability_permission_result`), or return a result before the permission check runs when an ability is executed directly from PHP (`wp_pre_execute_ability`; this includes WP-CLI's `wp ability run`, while the core REST run endpoint and the MCP Adapter check permissions before that filter can fire). Exposure is also per channel: the MCP Adapter follows `meta.mcp.public` or `public` and ignores `show_in_rest`, and WP-CLI applies no exposure check at all. Treat code that hooks those filters as part of the authorization surface and review it accordingly (see Benchmark §11.4). These behaviors were verified against WordPress 7.1.3 core, the `wp ability` WP-CLI command, and MCP Adapter 0.7.0.

- **Copyright and Compliance:** Monitor AI-generated content for copyright compliance and ensure that AI-assisted workflows align with organizational and legal disclosure requirements.
- **API Key Management:** Securely store and manage API keys for GenAI services. Never expose keys in client-side code and rotate them regularly. In WordPress 7.0, the Connectors API can source credentials from environment variables, PHP constants, or the database. Prefer environment variables or `wp-config.php` constants. Database-backed connector secrets are masked in the interface but not encrypted, so they should be treated as a lower-assurance fallback rather than the preferred operational model.

15. Additional Resources

15.1 WordPress Security Documentation

- Hardening WordPress — Advanced Administration Handbook
- WordPress Security White Paper (source)
- Brute Force Attacks (developer.wordpress.org)
- WordPress VIP Security Best Practices

15.2 Threat Intelligence and Industry Reports

- OWASP Top 10:2025 Web Application Security Risks
- Patchstack State of WordPress Security (annual whitepaper)
- Wordfence Annual WordPress Security Report
- Sucuri Website Threat Research Report
- Verizon Data Breach Investigations Report
- IBM X-Force Threat Intelligence Index

- IBM Cost of a Data Breach Report
- CISA Known Exploited Vulnerabilities Catalog

15.3 Standards and Frameworks

- NIST SP 800-63B: Digital Identity Guidelines — Authentication (Revision 4)
- NIST SP 800-61r3: Incident Response Recommendations and Considerations
- CIS Benchmarks
- ISO/IEC 27000: Information Security Management

15.4 Security Culture and Organizational Practices

- KnowBe4: Security Culture
- NIST: Users Are Not Stupid — Six Cyber Security Pitfalls Overturned

15.5 Deprecated and Invalid Constants Guardrail

Avoid documenting or implementing the following symbols as hardening controls:

- `FORCE_SSL_LOGIN` (deprecated)
- `DISALLOW_PLUGIN_EDITING` (not a core constant)
- `DISALLOW_PLUGIN_ACTIVATION` (not a core constant)
- `SECURE_LOGGED_IN_COOKIE` (not a core constant)
- `define( 'XMLRPC_REQUEST', false );` (`XMLRPC_REQUEST` is set internally during XML-RPC requests)

For Argon2 algorithm selection, reference the `wp_hash_password_algorithm` filter.

15.6 Cross-Document Control Classification Matrix

Use this matrix to keep this guide aligned with the Benchmark and Operations Runbook.

Control Area	Baseline	Optional Hardened	Environment-Specific
File editor/mods	`DISALLOW_FILE_EDIT = true`	`DISALLOW_FILE_MODS = true` with external update pipeline	Dashboard updates retained where platform-managed patch cadence requires it
REST API	Public content routes allowed; sensitive routes protected by auth/permissions	Block user-enumeration routes and harden custom endpoint callbacks	Global unauthenticated blocking only for private/intranet deployments
XML-RPC	Keep only when required by integrations	Block at the web server when unused (`xmlrpc_enabled` alone is partial: authenticated methods only)	Route/IP allowlisting for required integrations
File ownership	Documented least-privilege model per environment	Per-site process isolation and immutable deploy artifacts	Provider-constrained ownership with compensating controls
SSH access	Key-only auth + host firewall/fail2ban	Non-standard SSH port for scanner-noise reduction	Managed-host controls where SSH is unavailable

Related Documents

- **WordPress Security Benchmark** — Prescriptive, auditable hardening controls for the full WordPress stack (web server, PHP, database, application, file system). Use for compliance verification and configuration audits.
- **WordPress Security Style Guide** — Principles, terminology, and formatting conventions for writing about WordPress security. Use when producing vulnerability disclosures, customer communications, or documentation.

- **WordPress Operations Runbook** — Operational procedures template for WordPress sysadmins and SREs, covering deployment, maintenance, backup, incident response, and disaster recovery.
- **WordPress Security White Paper (WordPress.org, September 2025)** — The official upstream document describing WordPress core security architecture, maintained at wordpress.org/about/security/ (source repository).

License and Attribution

This document is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC-BY-SA-4.0). You may copy, redistribute, remix, transform, and build upon this material for any purpose, including commercial use, provided you give appropriate credit and distribute your contributions under the same license.